

Research Internship Assessment

NLP, Information Extraction, Knowledge Graphs & Applied AI

Climate–Health Intelligence Platform (CHIP) | FAST–NUCES

Assessment overview

- **Target candidates:** Senior undergraduate students and recent graduates in Computer Science, Data Science, or related fields.
- **Duration:** Strictly 62 hours from receipt of this assessment.
- **Use of LLMs:** Candidates *may* use LLMs (ChatGPT, Claude, Gemini, Llama, DeepSeek, or similar). All such use must be clearly documented — where, how, and for what purpose.
- **Structure:** Three tasks (two knowledge-graph construction tasks plus one real-time pipeline engineering task) plus one optional bonus challenge. Candidates are expected to attempt one of them.

Partial submissions are accepted. Candidates are not required to fully complete every task. A partial or incomplete submission is acceptable provided the following condition is met:

- Whatever state of the project exists in the GitHub repository at the deadline must be **deployable and runnable** using only the instructions in the README. A partially working pipeline that runs end-to-end for one source is preferable to a fully designed but non-functional system for five.

No commits after the deadline. The repository will be evaluated exactly as it stands at the submission deadline. Any commits, pushes, or force-pushes made after the deadline will be disregarded, and only the last commit timestamped before the deadline will be considered for evaluation. Candidates should plan their final commit with this in mind rather than relying on last-minute changes.

Important: The objective is *not* to simply prompt an LLM for answers. Candidates will be evaluated on pipeline design, data engineering skills, NLP methodology, knowledge graph construction, reproducibility, technical reasoning, and quality of insights. Strong solutions demonstrate engineering rigor beyond simple prompting.

1 Task 1 — Disease Intelligence Knowledge Graph

Background

A **sample document** — the District Health Information System (DHIS) Annual Report 2018 for Punjab, Pakistan, published by the Directorate General Health Services, Punjab — is provided to candidates purely as a reference to calibrate expectations for this task. It illustrates the kind of structured government health reporting candidates should expect: district-level case counts across 53 priority diseases (28 communicable, 25 non-communicable) with year-wise trends from 2013–2018, embedded charts and figures (bar, pie, line) reporting incidence rates and district rankings, facility-level data (THQ, DHQ, Teaching Hospitals, RHCs, BHUs), maternal and child health indicators by district, and the underlying DHIS-21/DHIS-22 reporting instrument templates that define the facility-level data schema.

The sample is illustrative only and is not the dataset to be processed for this task. The actual Task 1 dataset is a larger collection provided at the link below, and is expected to contain a broader range of documents and formats (e.g. multiple years, multiple report types, and/or multiple provinces) beyond the single sample report.

Dataset access: The actual Task 1 dataset is provided at the link below. Candidates must use this dataset — not the sample DHIS report alone — as the primary source for the Disease Intelligence Knowledge Graph.

https://drive.google.com/open?id=1mwXLmFHMh7htqM73trPE-eCig4r4TCwm&usp=drive_fs

Process the maximum data available. Candidates are expected to ingest and process as much of the provided dataset as feasible within the 62-hour window — not a small cherry-picked subset. Pipelines should be designed to generalize across all documents in the dataset (e.g. handling multiple years, districts, or report formats with the same extraction logic) rather than being hard-coded against a single file. If time constraints make full coverage impossible, candidates must clearly state in their technical report what proportion of the dataset was processed, which parts were excluded, and why — partial coverage with honest documentation is far preferable to silently processing only a convenient fraction of the data.

Candidates should expect tabular data embedded in prose, multi-panel figures with axis labels that must be read alongside surrounding text, and district-level structures that may vary across documents in the dataset. This is a realistic test of extracting structured intelligence at scale from real government health reporting, rather than from a single curated file.

Objectives

Build a complete pipeline that:

1. Extracts text, tables, and figure data (bar/pie/line chart values) from the report.

2. Identifies disease, district, facility, and indicator entities, including the 53 priority diseases and their communicable/non-communicable classification.
3. Resolves duplicate and aliased entities (e.g. district name variants such as “D.G Khan” vs “Dera Ghazi Khan”, “R. Y Khan” vs “Rahimyar Khan”, “M.B Din” vs “Mandi Bahauddin”).
4. Extracts relationships between diseases, districts, facilities, and time periods, including year-over-year trend relationships (2013–2018).
5. Normalizes temporal information to the **epidemiological week** as the primary unit (matching the native granularity of NIH/IDSR-style surveillance data), while also retaining calendar year/month for indicators that are only reported at coarser resolution (e.g. annual facility staffing, year-over-year trend summaries).
6. Constructs a disease intelligence knowledge graph linking districts, diseases, facilities, and indicators.
7. Generates analytical insights, including district-level rankings and communicable vs non-communicable disease patterns.

Required entity types

Entity type	Entity type
Disease (with Communicable/Non-Communicable flag)	Government Agency
Symptom / Complication	Geographic Location (District, Division)
Health Indicator (e.g. BOR, ALS, ANC-1 coverage)	Population Group (e.g. ANC attendees, <5 yrs)
Drug / Commodity (tracer drugs, FP commodities)	Date / Year
Health Intervention (immunization, FP visit, C-section)	Time Period (month, epidemiological year)
Health Facility (BHU, RHC, THQ, DHQ, Teaching Hospital)	

Required relationship types

Relationship	Meaning
REPORTED_IN	Disease case count reported in a district for a given year
TREATED_WITH	Disease or complication treated with a drug, commodity, or intervention
AFFECTS_GROUP	Disease or indicator affects a specific population group (e.g. <5 yrs, ANC-1 attendees)
MANAGED_BY	Facility or district managed by a government agency (e.g. DHIS MIS Cell)

Relationship	Meaning
LOCATED_IN	Health facility located within a district
OCCURRED_DURING	Case count or indicator value recorded during a specific year or month
PRECEDED_BY / FOLLOWED_BY	Year-over-year trend relationship between consecutive reporting periods (2013–2018)
RANKED_AGAINST	District incidence rate compared against other districts for the same disease

Minimum graph requirements

Metric	Minimum
Nodes	200
Relationships	500
Dataset coverage (proportion of provided files/documents processed)	As high as feasible; must be reported explicitly

These are floor values calibrated to the sample document; since the actual dataset is larger, candidates who process more of it are expected to comfortably exceed these minimums, and graph size will be interpreted relative to documented coverage rather than as an absolute pass/fail threshold.

Required analytical queries

The constructed graph should be able to answer:

1. Which districts had the highest incidence rate for each of the top five diseases?
2. Which diseases show the strongest year-over-year growth between 2013 and 2018?
3. Which districts consistently rank in the bottom quartile across multiple health indicators (e.g. low ANC-1 coverage, high stock-out rate, low deliveries at facilities)?
4. Which facility types (BHU, RHC, THQ, DHQ, Teaching Hospital) show the largest gap between sanctioned and filled staff positions?
5. Is there an association between communicable disease burden and facility-level indicators such as stock-out rate or staff vacancy rate in the same district?

2 Task 2 — Weather Intelligence Knowledge Graph

Background

Unlike Task 1, no dataset will be provided. Candidates must gather weather and climate information themselves using publicly available APIs.

Acceptable APIs (one or more may be used):

- OpenWeather API
- NOAA APIs
- NASA POWER API
- Meteostat API
- WeatherAPI
- Open-Meteo

Recommended API: Open-Meteo. Candidates are encouraged to use the Open-Meteo API as the primary data source for this task. It requires no API key or registration, offers a free historical archive endpoint (`archive-api.open-meteo.com`) suitable for retrospective analysis, and supports arbitrary latitude/longitude queries — making it straightforward to collect data for multiple countries without per-country credentials or rate-limit complications. Candidates using a different API from the acceptable list should justify the choice in their technical report.

Geographic scope requirement: Pakistan and neighbouring countries. Weather and climate data must be collected not only for Pakistan but also for its immediate neighbours — **India, Afghanistan, Iran, and China (Xinjiang/western region)**. Regional weather systems (e.g. monsoon patterns, western disturbances, upstream river-basin precipitation) frequently originate outside Pakistan's borders before affecting it. Including neighbouring countries allows the graph to capture cross-border climate patterns and precursor signals that may help explain or anticipate weather-related events inside Pakistan. At minimum, candidates should collect comparable weather entities (rainfall, temperature, extreme events) for a representative set of locations in each neighbouring country, not just Pakistan.

Objectives

Build an automated pipeline that:

1. Collects weather data from public APIs for Pakistan and its neighbouring countries (India, Afghanistan, Iran, China).
2. Creates structured weather events, each tagged with its country and location.
3. Extracts and normalizes weather entities across all collected countries on a consistent schema.
4. Identifies significant climate events, including those in neighbouring countries that may precede related events in Pakistan.

- Constructs a weather intelligence knowledge graph capturing both within-country and cross-border relationships.

Required entity types

Entity type	Entity type
Rainfall Event	Wind Event
Temperature Event	Climate Indicator
Heatwave	Location (city/district level)
Drought	Country (Pakistan, India, Afghanistan, Iran, China)
Flood	Date
Storm	Time Window

Required relationship types

Relationship	Meaning
OCCURRED_IN	Weather event occurred in a specific location
CAUSED	Weather event caused another event or outcome
PRECEDED	Event temporally preceded another event
FOLLOWED	Event temporally followed another event
ASSOCIATED_WITH	Event associated with a climate indicator or pattern
AFFECTED	Event affected a location or population
UPSTREAM_OF	Weather event in a neighbouring country precedes or contributes to a related event in Pakistan (e.g. a western disturbance over Afghanistan/Iran preceding rainfall in Pakistan)

Minimum graph requirements

Metric	Minimum
Nodes (across Pakistan + neighbours)	200
Relationships	350
Countries represented (of Pakistan, India, Afghanistan, Iran, China)	4 or more

Required analytical queries

- Which districts experienced the highest rainfall?
- Which regions experienced multiple extreme weather events?

3. What weather patterns frequently occur together?
4. Which climate indicators show increasing trends?
5. Which districts appear most vulnerable to extreme weather?
6. Do extreme weather events in neighbouring countries (e.g. heavy rainfall in upstream Indian or Afghan regions) precede related events in Pakistan, and by what typical time lag?

3 Task 3 — Real-Time Multi-Source Data Integration (Producer–Consumer Pipeline)

Background

Tasks 1 and 2 focus on building knowledge graphs from largely static or self-collected data. Task 3 shifts focus to **systems engineering**: candidates must design and implement a real-time, distributed data pipeline that ingests and harmonizes heterogeneous streams using a producer–consumer architecture built on **Apache Kafka and Apache Spark** — the same NAaaS-style distributed stream-processing stack used in the CHIP platform.

Where Tasks 1 and 2 test extraction and graph construction, Task 3 tests whether a candidate can design topic schemas, write fault-tolerant producers and consumers, and harmonize multiple disparate formats (batch PDFs, near-real-time JSON, event-driven alerts, periodic policy documents, and continuous RSS feeds) into a single normalized schema, at scale.

Objectives

Design and implement a real-time data pipeline that ingests, harmonizes, and integrates heterogeneous datasets spanning health, meteorology, disasters, climate policy, and media. The pipeline should extend a Kafka + Spark architecture to support multi-source ingestion and cross-domain entity linking.

Required input sources

Source	Data	Ingestion mode
NIH Pakistan — Weekly Surveillance Reports	Epidemiological data: dengue, diarrhoea, influenza-like illness, malaria incidence, IDSR reporting compliance	Batch-style ingestion converted into Kafka topics
PMD — Meteorological Data	Temperature, precipitation, humidity, heatwave/flood alerts	Near real-time JSON/API feeds
NDMA / PDMA — Disaster Situation Reports	Floods, landslides, droughts, disaster alerts	Event-driven ingestion
Ministry of Climate Change — Policy & Environmental Data	Climate and environmental policy updates	Periodic ingestion (monthly/quarterly)
Media Streams — National & Regional News	Curated headlines from Dawn, Express Tribune, The News, etc.	Continuous ingestion via RSS/JSON feeds

Relationship to Tasks 1 and 2: NIH and PMD data sources overlap with the earlier tasks, but the objective here is different — Task 3 evaluates the *pipeline and infrastructure*, not the knowledge graph itself. Candidates may reuse parsing logic developed for Task 1 where appropriate, but must wrap it inside a proper Kafka producer rather than a one-off script.

Required components

- **Kafka producers** — one producer per source, each publishing to a dedicated topic with an explicit schema.
- **Spark Structured Streaming consumers** — stream processors that consume, normalize, and harmonize data across topics.
- **Normalization:**
 - Temporal tagging with HeidelbergTime.
 - Geocoding with GeoPy + OpenStreetMap.
- **Entity linking** — align outbreak events, environmental anomalies, and disaster alerts with administrative regions (district/division level), enabling cross-domain joins.
- **Storage layer** — persist harmonized data in Hadoop (HDFS) or Parquet for downstream analytics.
- **Monitoring** — basic logging/metrics for ingestion success and failure per source.

Temporal normalization standard: epidemiological week, not calendar month

The canonical temporal unit for this pipeline is the epidemiological week (W1–W52), not the calendar month. This is a deliberate design requirement, not a stylistic choice, and is based directly on the structure of NIH’s actual published bulletins:

- Every disease case count, district table, IDSR compliance figure, and trend chart in the NIH Public Health Bulletin is indexed by epidemiological week (e.g. “Week 30, 2024”), with multi-week comparisons shown as W28/W29/W30 groupings — never by calendar month.
- Week-over-week trend charts in the bulletin (e.g. ILI or AD case trends) span quarter boundaries using a W1–W52 axis that does not reset at month boundaries, so converting to monthly buckets would require an arbitrary re-aggregation not supported by the source data.
- Outbreak investigations (e.g. a cholera cluster) are reported and acted on at day/week granularity; collapsing this to monthly resolution would destroy the signal an early-warning system depends on.

Practical requirement: the Spark consumers for NIH and disaster/event-driven sources (NDMA/PDMA) must tag and key records by epidemiological week. PMD data (which arrives at finer, near-real-time granularity) should be aggregated *up* to the matching epidemiological week when joined against NIH data, rather than NIH data being collapsed down to monthly. Only the Ministry of Climate Change policy stream, which is inherently periodic (monthly/quarterly), should retain a month-level timestamp — candidates should not force-fit this source onto a weekly key, and should instead design the harmonized schema to tolerate mixed temporal resolutions across topics, joining at the coarsest common level only when a query specifically requires it.

Deliverables

1. **Codebase:** Kafka producers, Spark streaming jobs, normalization and entity-linking logic.
2. **Architecture diagram:** showing data flow from sources → Kafka → Spark → harmonized output.
3. **README:** instructions to run the pipeline, assumptions made, and dependencies.
4. **Sample output:** a harmonized dataset (e.g. Parquet/CSV) demonstrating integration across at least two sources.

Evaluation criteria for this task

- Correctness of ingestion and schema normalization.
- Ability to handle heterogeneous formats (CSV, JSON, RSS, API).
- Implementation of temporal tagging at the correct granularity (epidemiological week for surveillance/event data; month for inherently periodic policy data) and geocoding.
- Quality of entity linking across domains.
- Code readability, modularity, and documentation.
- Demonstrated understanding of real-time vs. batch ingestion trade-offs.

Scope note: A fully production-grade pipeline covering all five sources end-to-end within 62 hours is not expected. Candidates should prioritize a working, well-architected pipeline for at least *three* sources (including at least one batch and one streaming/event-driven source) over a superficial attempt at all five. Document clearly in the README which sources were fully implemented, partially implemented, or stubbed out, and why.

4 Deliverables

1. GitHub repository

Candidates must submit a GitHub repository containing:

1. Source code
2. README (including the video demo link — see Deliverable 3)
3. Data processing scripts
4. Knowledge graph construction code (Tasks 1 and 2)
5. Kafka producers, Spark streaming jobs, and entity-linking logic (Task 3)
6. Documentation
7. Reproducibility instructions

2. Technical report (maximum 12 pages)

The report must contain:

- System architecture (including the Task 3 pipeline diagram)
- Data sources
- Entity extraction methodology
- Entity resolution strategy
- Knowledge graph schema
- Example graph visualizations
- Pipeline design decisions for Task 3 (topic schema, partitioning, real-time vs. batch trade-offs)
- Analytical findings
- Challenges encountered
- Use of LLMs

3. Video demo (required, 5–10 minutes)

All candidates must submit a single screen-recorded video, **5–10 minutes long**, covering the following:

- A brief walkthrough of the overall solution architecture (1–2 minutes).
- A live run of the Task 1 pipeline against the dataset, showing extraction output and a view of the resulting knowledge graph (e.g. in Neo4j Browser or an equivalent graph viewer).
- A live run of the Task 2 pipeline, showing the weather data collection step and the resulting graph, including at least one location from a neighbouring country.
- A live run of the Task 3 pipeline, showing at least one Kafka producer publishing and the corresponding Spark consumer processing it in real time, plus a look at the harmonized output.
- Answers to at least two of the required analytical queries from Task 1 or Task 2, run live against the constructed graph (not pre-computed screenshots).
- A brief closing summary of what was completed, what was partial or skipped, and why.

Format: Screen recording with narration (voice-over or captions), uploaded to an unlisted YouTube link, Google Drive link, or Loom link, and referenced in the README. The video must reflect the exact codebase state in the final pre-deadline commit — recordings demonstrating a later or different version of the code will not be considered valid.

5 Evaluation Criteria

Criterion	Weight
Information Extraction Accuracy & Dataset Coverage (Tasks 1–2)	15%
Entity Resolution Quality (Tasks 1–2)	10%
Relationship Extraction Quality (Tasks 1–2)	10%
Knowledge Graph Design (Tasks 1–2)	10%
Pipeline Architecture & Producer–Consumer Design (Task 3)	15%
Normalization, Entity Linking & Cross-Domain Integration (Task 3)	10%
Code Quality and Reproducibility (All Tasks)	8%
Documentation Quality (All Tasks)	7%
Video Demo — Clarity, Coverage & Live Functionality	5%
Analytical Insights (Tasks 1–2)	5%
Innovation and Bonus Challenge	5%
Total	100%

Final reminder: Strong solutions demonstrate engineering rigor — not just LLM prompting. Document every instance of LLM assistance clearly, including the tool used, the purpose, and the extent of reliance on its output.